

Extending NFA simulation reduction

Samuel Lupták*

Abstract

The problem of reducing deterministic finite automata to their minimal form is a well-researched one. In some cases, it may be beneficial to try and reduce their non-deterministic versions, although this problem is not as easy and straight-forward. The problem can be solved by computing a simulation relation between states which, is then used for the reduction. The reduction is then performed by merging together states, which reduces the overall number of states and possibly transitions while perserving the language of the automaton. The goal of this paper is to implement additional rules by which states can be merged together into the Mata library developed at the VeriFit research group. Before our work, Mata had only one merging rule. We have added another two merging rules as well as one iterative reduction method called simulation fixpoint. We also show an extensive comparison of NFA reduction methods, which shows the pros and cons of the different algorithms.

Keywords: Nondeterministic automata — Reduction — Simulation relation

Supplementary Material: [Mata library](#) — [Implementation](#) — [Benchmark](#)

*xluptas00@stud.fit.vutbr.cz, Faculty of Information Technology, Brno University of Technology

1. Introduction

Deterministic finite automata (DFA) are a formal mathematical model used in many applications today, including formal verification, compilers, pattern matching and even fields beyond informatics such as biology. In practical applications, determinism is a well-needed property of these automata because non-deterministic finite automata (NFA) are harder, sometimes even impossible, to implement. In contrast, when designing a finite automaton, it is easier to construct a NFA because it allows more freedom in its construction and its size can be a lot smaller.

In some applications, like formal program verification, even NFAs may have an unpleasant size making determinization computationally hard, because of the potential explosion in the number of states of the final DFA [4]. In these situations, the reduction of NFA becomes useful. Unfortunately, NFA reduction is not as easy as DFA reductions. NFA reduction is proven to be a PSPACE complete problem [4], so the optimal solution is very hard to find. That is why heuristics are used instead, like construction of the simulation relation, to partially solve this problem. The main requirements for these heuristics are speed and the number of re-

duced states. We have implemented another heuristic which extend the already existing simulation heuristic in Mata, so we will be interested in the number of resulting states as well as the time its takes to reduce a given automaton.

To reduce a NFA using simulations the simulation relation must be computed (this is the hard part of computation) and then we can use this relation to reduce the input automaton. The reduction is done by using the simulation relation to merge states, which reduces the number of states and possibly transitions in the resulting automaton while preserving the language of the input automaton. States must be merged according to specific rules in order not to change the language of the output automaton.

As mentioned above, Mata [2] already has one of the merging rules implemented, in order to be able to reduce NFAs using simulations. We have experimented with two other rules, which will be fully explained later, and also with an iterative method of applying these rules to achieve even higher reduction. The additional rules were proposed by L. Ilie, G. Navarro, and S. Yu in their paper *On NFA Reduction* [5].

We have extensively benchmarked our new reduction rules not only with the original simulation reduction, but as well with the other NFA reduction algorithms thus creating a complex benchmark of NFA reduction algorithms. The automata sample consisting of 26158 automata that were used for the benchmarks can be found in the supplementary material (*automata* folder inside *Benchmark*). One special benchmark was also conducted, which compared the new simulation reduction with the residual [3] reduction on a specific sample of automata used in Z3 Noodler [1] also developed at VeriFit research group. This special benchmark was conducted in order to see which reduction method would be more suitable for this kind of job.

2. Preliminaries

Let us begin with some formal definitions that need to be understood so we can explain merging rules in detail.

Definition 1 An *alphabet* is a non-empty finite set whose elements are called letters. It is denoted with the greek letter Σ .

Definition 2 A *string* is a sequence of letters $a_0a_1 \dots a_n$ over an alphabet Σ such that $\forall i \leq n : a_i \in \Sigma$. The empty string is denoted ε .

Definition 3 Let Σ be an alphabet, then Σ^* is the set of all strings over Σ . Every $L \subseteq \Sigma^*$ is called a *language* over Σ .

Next we define a non-deterministic finite automaton. We will use this definition extensively in the following text.

Definition 4 A Non-deterministic finite automaton M , denoted as *NFA* M , is defined as a quintuple $M = (Q, \Sigma, \delta, S_0, F)$ where:

- Q is a finite set of states,
- Σ is an alphabet,
- $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ is a transition function,
- $S_0 \subseteq Q$ is a set of initial states, and
- $F \subseteq Q$ is a set of final states.

Definition 5 Let $M = (Q, \Sigma, \delta, S_0, F)$ be an NFA. The *Reverse NFA* M^R with respect to M is the quintuple $M^R = (Q, \Sigma, \delta^R, F, S_0)$ where δ^R is a function such that $\forall p, q \in Q : \forall a \in \Sigma : p \in \delta(q, a) \Leftrightarrow q \in \delta^R(p, a)$.

Notice that the only differences between the NFA M and the NFA M^R are that the initial and final states are swapped and all the transitions are reversed.

2.1 Reduction using simulations

The basic idea of reducing the number of states of a NFA with simulations is to build a preorder relation between the states. Let us define the properties that a preorder relation must satisfy.

Definition 6 Let R be a binary relation. R is a *preorder* if it is:

- **Reflexive** $\forall a : aRa$,
- **Transitive** $\forall a, b, c : aRb \wedge bRc \implies aRc$.

Let us specify when two states are in a simulation relation.

Definition 7 Let $M = (Q, \Sigma, \delta, S_0, F)$ be an NFA. $\preceq_R$ is a preorder relation which satisfies the following properties:

- $P_1 : \preceq_R \cap (F \times (Q - F)) = \emptyset$,
- $P_2 : \forall p, q \in Q : \forall a \in \Sigma : (q \preceq_R p \implies \forall q' \in \delta(q, a), \exists p' \in \delta(p, a), q' \preceq_R p')$.

Relation $\preceq_L$ is defined similarly, but instead of M we use the reversed automaton M^R . Now we define when and how we can merge states.

Definition 8 Let $M = (Q, \Sigma, \delta, S_0, F)$ be an NFA. Let us construct the largest $\preceq_R$ and $\preceq_L$ over M . We can merge two states $p, q \in Q$ if one of the following rules hold:

- $R1 : q \preceq_R p \wedge p \preceq_R q$,
- $R2 : q \preceq_L p \wedge p \preceq_L q$,
- $R3 : q \preceq_R p \wedge q \preceq_L p$,

Definition 9 Let $M = (Q, \Sigma, \delta, S_0, F)$ be an NFA. When we *merge* two states $p, q \in Q$ according to some rule (from definition 8) in M we modify M such that:

- Remove p from Q and F if it is a final state,
- All $t \in \delta$ in the form of $t = (x, y, p)$ or $t = (p, x, y)$ are modified respectively as $t = (x, y, q)$ and $t = (q, x, y)$.

It is now obvious that when we merge two states, the total state count of a given NFA decreases thus **reducing** the size of a given automaton.

Example 1 Let us give an example of a reduction using $R1$ and $R2$. Figure 1 is the unreduced automaton. Figure 2 and Figure 3 are the reduced automata according to $R1$ and $R2$ respectively.

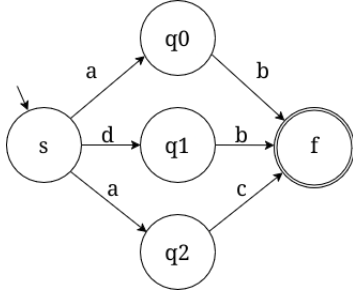


Figure 1. NFA M_1

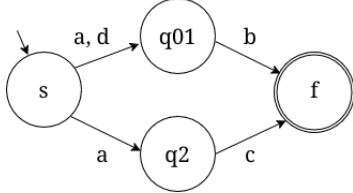


Figure 2. Reduced M_1 using R1

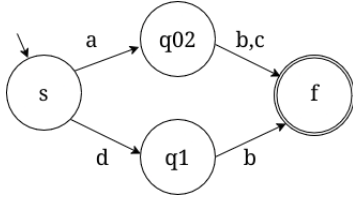


Figure 3. Reduced M_1 using R2

3. Implementation

The library to which we have implemented these merging rules is called Mata². This library is being developed at BUT FIT at the VeriFit research group. Mata is a C++ fast and simple automata library, which supports many operations on NFAs including language concatenation, union, inclusion checking and more, including simulation computation and reduction.

3.1 Algorithms

As mention before, Mata already has one of these merging rules implemented. The implemented rule is the R1 rule. The main goal of this work is to implement rules R2 and R3 into Mata. It is also important to note that Mata currently supports only the computation of the forward simulation $\preceq_R$. This technicality can be bypassed by computing the reversed automaton.

Algorithm 1: Implementation of the merging rule R2

Input: NFA $\mathcal{A} = (Q, \Sigma, \delta, I, F)$
Output: Reduced NFA $\mathcal{A}$

```

1  $M \leftarrow \text{reverse}(\mathcal{A})$ 
2  $\preceq_R \leftarrow \text{compute } \preceq_R(M)$ 
3  $M' \leftarrow R1(M, \preceq_R)$ 
4 return  $\text{reverse}(M')$ 

```

²<https://github.com/VeriFIT/mata>

Rule R2 is implemented as Algorithm 1 outlines.

The functions are interpreted as $\text{reverse}(M)$ computing M^R from M , $\text{compute } \preceq_R(M)$ computing the forward simulation $\preceq_R$ on M and $R1(M, \preceq_R)$ computing the reduced automaton from M with relation $\preceq_R$ using the rule R1.

Implementation of R3 is not that simple as R2. To solve this problem we have implemented an entirely new merging algorithm, that is too complex to overview in this report. The algorithm can be seen in the *implementation* section of the supplementary material. We assume that the forward simulation $\preceq_R$ and backward simulation $\preceq_L$ have already been computed as the algorithm needs them as inputs.

3.2 Simulation fixpoint

The overview of the simulation fixpoint algorithm is shown in Algorithm 2.

Algorithm 2: Simulation fixpoint implementation

Input: NFA $\mathcal{A} = (Q, \Sigma, \delta, I, F)$
Output: Reduced NFA $\mathcal{A}$

```

1  $M \leftarrow \mathcal{A}$ 
2  $\text{Changed} \leftarrow \text{True}$ 
3 while  $\text{Changed}$  do
4    $\text{Changed} \leftarrow \text{False}$ 
5    $\preceq_R \leftarrow \text{compute } \preceq_R(M)$ 
6    $\preceq_L \leftarrow \text{compute } \preceq_L(M)$ 
7    $M' \leftarrow R3(M, \preceq_R, \preceq_L)$ 
8   if  $\text{same}(M, M')$  then
9      $\text{Changed} \leftarrow \text{True}$ 
10   $\preceq_R \leftarrow \text{compute } \preceq_R(M')$ 
11   $M'' \leftarrow R1(M', \preceq_R)$ 
12  if  $\text{same}(M', M'')$  then
13     $\text{Changed} \leftarrow \text{True}$ 
14     $\preceq_L \leftarrow \text{compute } \preceq_L(M'')$ 
15     $M''' \leftarrow R2(M'', \preceq_L)$ 
16    if  $\text{same}(M'', M''')$  then
17       $\text{Changed} \leftarrow \text{True}$ 
18     $M \leftarrow M'''$ 
19 return  $M$ 

```

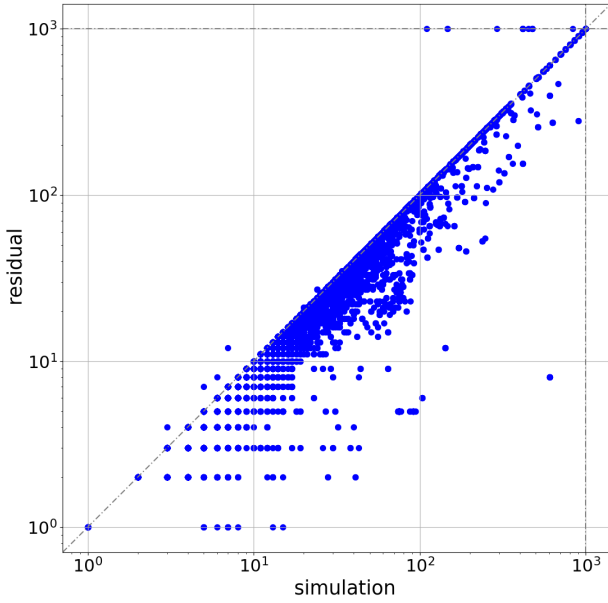


Figure 4. Number of states comparison [States]

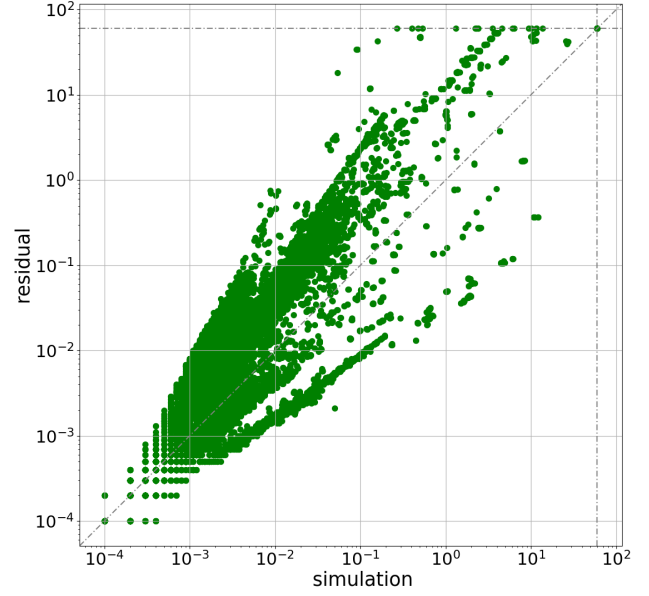


Figure 5. Time comparison [s]

As can be seen in Algorithm 2, this is an iterative algorithm combining all of the merging rules. It uses three control boolean variables that are set to *True* whenever the automaton is the same before and after the reduction. When all of these control variables are set to *True* the algorithm is terminated and the last version of the reduced automaton is returned. The function $\text{same}(M, M')$ compares automata M and M' and if they are equal (they have the same states and transitions), it returns *True*.

4. Experimental evaluation

Part of this work is to compare not only the different versions of simulation reduction but also compare different reduction algorithms. The algorithms can be divided into two categories: *Minimization* algorithms (Hopcroft and Brzozowski) and *Reduction* algorithms (Residuals and Simulations). *Minimization* takes a NFA on the input and produces a language equivalent (canonical) DFA on the output. *Reduction* on the other hand takes a NFA on the input and produces a language equivalent NFA on the output (the resulting NFA might NOT be in the minimal form).

As the comparison between *Minimization* and *Reduction* algorithms are not very useful we will not discuss them in this report. A report containing all of the graphs is available upon request.

We start by showing two graphs that compare residual reduction with the simulation reduction using only R1. Figure 4 shows the comparison of states in the reduced automaton, where we see that residuals win. Figure 5, on the other hand, shows the time needed for the reduction and here, simulations win.

This time to number-of-states trade-off is the main

element in choosing which reduction algorithm to use, as different applications require different qualities from the reduction algorithm. One of these applications is the Z3 Noodler tool [1], which, when experimentally evaluated, showed better results with residuals than with simulations.

In Figure 6 we show the improvement in state reduction when using fixpoint as compared to simulation using only R1. The graph looks better than the improvement actually is as the number of automata that have improved state reduction is only about 3000. Figure 7 shows the comparison between fixpoint and simulation using only R1 in terms of time. We can see that there is an exponential time increase (graph is in logarithmic scale), which is not optimal as this time

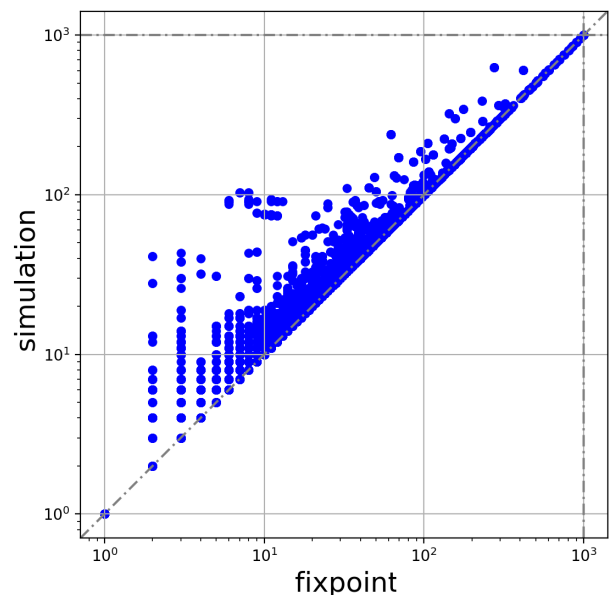


Figure 6. Number of states comparison [States]

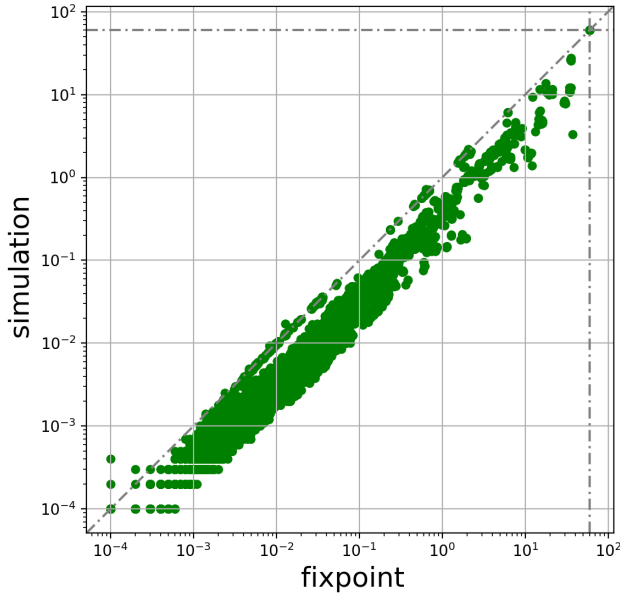


Figure 7. Time comparison [s]

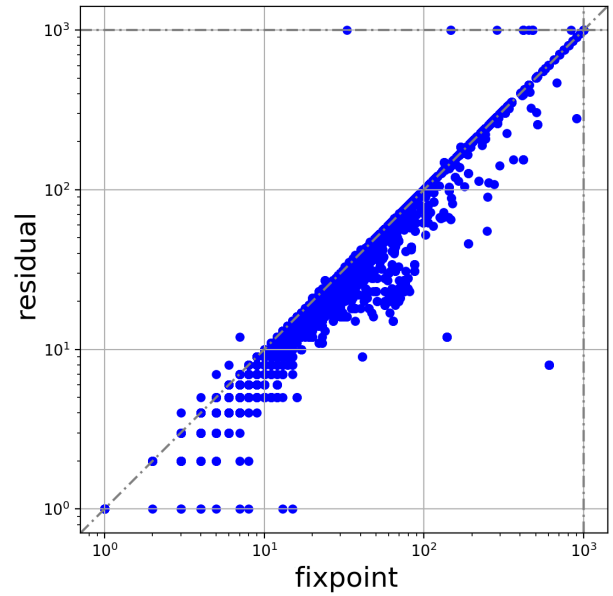


Figure 8. Number of states comparison [States]

increase is affecting every automata, not just the ones who have better state reduction.

Figure 8 shows the number of states after reduction between fixpoint and residuals, which does not show a marginal improvement. If we consider the fact that fixpoint is much slower than the simulation with R1, we take the only advantage over residuals, which is not very useful.

5. Conclusion

We have implemented two new merging rules to Mata with one iterative heuristic called fixpoint for reduction. The new rules show great potential, but fixpoint (implemented as it is now) might not be the right usage.

As a future work, it is possible to improve the fixpoint heuristic to only compute the forward and backward simulation once, and then just modify it after every reduction, which will improve the time needed for reduction significantly, but also it will increase the number of states after reduction because the modifications to the already computed simulations do not guarantee their coarseness.

The results of the benchmarks sparked a conversation between the research group members, which lead to the testing of simulations versus residuals in the Noodler tool.

6. Thanks

I thank Ondřej Lengál for the opportunity to work on this project, for his endless patience with me and for the great advices he gave me on our weekly consultations. I also thank David Chocholatý because he gave me an excellent detailed introduction to Mata

and Tomáš Kocourek for our occasional conversations about simulations.

References

- [1] CHEN, Y.-F., CHOCHOLATÝ, D., HAVLENA, V., HOLÍK, L., LENGÁL, O. et al. Z3-Noodler: An Automata-based String Solver. In: FINKBEINER, B. and KOVÁCS, L., ed. *Tools and Algorithms for the Construction and Analysis of Systems*. Cham: Springer Nature Switzerland, 2024, p. 24–33. ISBN 978-3-031-57246-3.
- [2] CHOCHOLATÝ, D., FIEDOR, T., HAVLENA, V., HOLÍK, L., HRUŠKA, M. et al. Mata: A Fast and Simple Finite Automata Library. In: *Tools and Algorithms for the Construction and Analysis of Systems*. Springer Nature Switzerland, 2024. ISBN 978-3-031-57249-4.
- [3] DENIS, F., LEMAY, A. and TERLUTTE, A. Residual Finite State Automata. In: FERREIRA, A. and REICHEL, H., ed. *STACS 2001*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, p. 144–157. ISBN 978-3-540-44693-4.
- [4] ESPARZA, J. and BLONDIN, M. *Automata Theory: An Algorithmic Approach*. 1st ed. The MIT Press, 2023. ISBN 9780262048637.
- [5] ILIE, L., NAVARRO, G. and YU, S. On NFA Reduction. In: *Theory Is Forever: Essays Dedicated to Arto Salomaa on the Occasion of His 70th Birthday*. Springer Berlin Heidelberg, 2004. ISBN 978-3-540-27812-2.